OS Orange -4 Lab Report: PES-VCS

Name: Praveen
SRN: PES1UG25CS832
Assignment: OS Orange -4 Platform:
Ubuntu 22.04

1. Objective

This assignment implements a local version control system (PES-VCS) with:

- content-addressable object storage,
- tree snapshot construction,
- staging index management,
- commit creation and history traversal,
- verification through unit and integration tests.

2. Implementation Summary

Implemented all required TODO functions:

- object.c
  - object_write
  - object_read
- tree.c
  - tree_from_index
- index.c
  - index_load
  - index_save
  - index_add
- commit.c
  - commit_create

3. Screenshot Evidence

1A. Phase 1 Unit Test (./test_objects)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1A Phase 1 test_objects

1B. Sharded Object Storage (find .pes/objects -type f)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

1B Sharded object files

2A. Phase 2 Unit Test (./test_tree)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2A Phase 2 test_tree

2B. Raw Tree Object Binary Format (xxd … | head -20)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ xxd .pes/objects/e0/0c50e16a2df38f8d6b
f809e181ad0248da6e6719f35f9f7e65d6f606199f7f | head -20
00000000: 626c 6f62 2036 0077 6f72 6c64 0a         blob 6.world.
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$
```

2B Raw tree xxd

3A. pes init + pes add + pes status

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ ./pes init

echo "hello" > file1.txt
echo "world" > file2.txt

./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  index.c
  untracked:  Makefile
  untracked:  pes.c
  untracked:  test_tree
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  test_sequence.sh
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  hello.txt
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  object.c
  untracked:  pes.h
  untracked:  commit.c
  untracked:  README.md

praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$
```

3A Phase 3 init add status

3B. Index Text Format (cat .pes/index)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776430729 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776430729 6 file2.txt
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$
```

3B Phase 3 index text

4A. Commit History (./pes log with three commits)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 84e0b5c61b1f... Initial commit
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ ./pes log
commit 84e0b5c61b1fa467166c2dc599f3d51d2cbc77e841aa5b2742f4908d531e0805
Author: PES User <pes@localhost>
Date:   1776431230

    Initial commit
```

4B. Object Store Growth (find .pes -type f | sort)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/84/e0b5c61b1fa467166c2dc599f3d51d2cbc77e841aa5b2742f4908d531e0805
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/refs/heads/main
```

4C. Reference Chain (cat .pes/refs/heads/main and cat .pes/HEAD)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
84e0b5c61b1fa467166c2dc599f3d51d2cbc77e841aa5b2742f4908d531e0805
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$
```

4C Phase 4 refs and HEAD

Final Integration Test (make test-integration)

```
praveen@praveen-Inspiron:~/OS_orange_4/PES1UG25CS832-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

```
commit 0cf52997332594c56052fe3081759a3fdc2e496bc96943a990bf27997be33d0f
Author: PES User <pes@localhost>
Date:   1776431453

    Add farewell

commit 1c7622712ba41bcef34062d5fa29e0e9a901109feb85e08effc2aa9732c1e5ea
Author: PES User <pes@localhost>
Date:   1776431453

    Update file.txt

commit 5420771e46e326337b3df3dc6ebc1e367a14aa1cafb4f92829ef5fae3772e4ec
Author: PES User <pes@localhost>
Date:   1776431453

    Initial commit
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
0cf52997332594c56052fe3081759a3fdc2e496bc96943a990bf27997be33d0f

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/0c/f52997332594c56052fe3081759a3fdc2e496bc96943a990bf27997be33d0f
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1c/7622712ba41bcef34062d5fa29e0e9a901109feb85e08effc2aa9732c1e5ea
.pes/objects/54/20771e46e326337b3df3dc6ebc1e367a14aa1cafb4f92829ef5fae3772e4ec
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Final integration output part 1

Final integration output part 2

4. Analysis Answers

Q5.1: How would pes checkout work?

A branch is a ref file in .pes/refs/heads storing a commit hash. To

implement checkout:

1. Resolve the target commit hash from .pes/refs/heads/.
2. Update .pes/HEAD to point to ref: refs/heads/ (or hash for detached checkout).
3. Read target commit -> target tree.
4. Rewrite working directory to match target tree:
   - create/update files from blobs,
   - create/remove directories,
   - apply modes (100644 or 100755),
   - remove tracked paths not present in target tree.
5. Rewrite .pes/index so staged state matches checked-out snapshot.

Why complex:

- Must safely coordinate HEAD, index, and working directory updates. • Must avoid overwriting local uncommitted work.
- Must handle deletes, nested trees, and mode changes atomically.

Q5.2: Detect dirty working directory conflicts using index + object store

Use three snapshots:

- A: index entries (path -> staged blob hash + metadata) • B: current HEAD tree (from object store)
- C: target branch tree (from object store) Conflict

detection:

1. For each tracked path in union(A, B), inspect working directory file.
2. If missing or metadata differs from index entry, confirm by hashing file and comparing with A[path].hash.
3. Mark path dirty if working file != staged/index version.
4. If dirty path would change between current and target (B[path] != C[path], or present/deleted mismatch), refuse checkout.

This prevents local changes from being overwritten by branch switching.

Q5.3: Detached HEAD behavior and recovery

Detached HEAD means .pes/HEAD stores a commit hash directly, not a branch ref.

If commits are made in detached state:

- commits are created normally, • parent chain is valid,
- but no branch pointer advances.

These commits can become unreachable once HEAD moves away.

Recovery:

- create a branch ref pointing to that detached commit hash (write hash into .pes/refs/heads/) before GC.

Q6.1: Algorithm for garbage collection of unreachable objects

Use mark-and-sweep.

Mark phase:

1. Start from all refs in .pes/refs/heads/* (and detached HEAD hash, if any).
2. Traverse commits recursively:
   - mark commit hash,
   - traverse parent,
   - traverse commit tree.
3. Traverse tree objects recursively:
   - mark tree hash,
   - mark blob hashes,
   - recurse into subtree hashes.

Sweep phase:

1. Scan all .pes/objects// files.
2. Convert path to object hash.
3. Delete objects not in reachable set.

Data structure:

- hash set for reachable hashes (O(1) expected lookup).
- stack/queue for DFS/BFS traversal.

Scale estimate for 100,000 commits and 50 branches:

- typically near unique reachable history, not 100,000 x 50,
- about 100,000 commit objects plus corresponding reachable trees/blobs,
- usually hundreds of thousands to a few million total reachable objects depending on file churn.

Q6.2: Why concurrent GC with commit is dangerous

Race example:

1. Commit process writes new blobs/trees first.
2. Before commit ref update, these objects are temporarily unreachable from refs.
3. Concurrent GC marks from current refs and misses those new objects.
4. GC sweep deletes them.
5. Commit object is then written/updated to reference deleted objects.

Result: broken commit graph with missing objects.

How Git avoids this:

- uses locking and coordinated updates around refs,

- uses conservative pruning windows (do not immediately prune recent loose objects), • manages pack/visibility safely to avoid deleting in-flight objects.

5. Notes

- Integration test script path issue with spaces was handled by running from a no-space symlink path:
  - /tmp/pesvcs-nospace/repo
- This preserves provided files unchanged while allowing successful execution.